

CPIT-380 Multimedia Technologies – group project

MultiMedia Processing Studio

Prepared By:

Azzam Alshamrani

Hossam Alzharani

Mohammed Khalid Alahmadi

Abdulaziz Alaa Alorabi

Wael alsulami

Date of Submission

14 May 2026

Table of Contents

Introduction	3
Problem Definition	3
Project Objectives.....	4
Background Research	5
Tools and Libraries Used.....	5
1. CustomTkinter	6
2. Pillow	6
3. NumPy	6
4. Wave Module	6
5. PyDub	6
6. OpenCV.....	6
7. FFmpeg	6
System Methodology.....	7
System Design.....	7
Implementation	8
User Manual	8
Testing and Results	9
Table 1: System Testing Results.....	10
Testing Screenshots	10
Challenges and Limitations	13
Conclusion.....	14
References	15

Introduction

Multimedia technologies are widely used in modern applications because they allow computers to process and manipulate different types of media such as images, sounds, text, and videos. These technologies are used in many fields including education, entertainment, healthcare, security, and digital content creation. This project, titled MultiMedia Processing Studio, is a desktop-based multimedia processing application developed as a group project for CPIT-380 Multimedia Technologies. The system provides a simple graphical user interface that allows users to upload multimedia files, apply different processing operations, preview the results, and save the processed output. The project focuses on demonstrating important multimedia concepts through practical implementation. It includes image processing, color transformations, geometric transformations, audio processing, and video-frame processing. The main goal of the project is not to build professional editing software, but to create an educational tool that shows how multimedia data can be processed at the pixel, sample, and frame level.

Problem Definition

Many students learn multimedia processing concepts theoretically, such as changing image brightness, applying color transformations, modifying audio samples, and processing video frames. However, these concepts can be difficult to fully understand without a practical application that shows how media data is processed step by step. Most professional editing tools allow users to apply filters and effects, but they usually hide the actual processing steps from the user. As a result, it becomes difficult for students to understand how pixels, audio samples, and video frames are manipulated internally.

This project addresses this problem by developing a simple educational multimedia processing application. The system allows users to upload images, audio files, and videos, apply different processing operations, preview the results, and save the output. The main purpose is to demonstrate CPIT-380 multimedia concepts through a working graphical user interface and manual processing operations.

Project Objectives

The main objective of this project is to develop an educational multimedia processing application that demonstrates important CPIT-380 concepts through a practical graphical user interface.

The specific objectives of the project are:

1. To build a user-friendly desktop application that allows users to upload and process multimedia files.
2. To apply image processing operations such as brightness adjustment, contrast adjustment, histogram equalization, average filter, median filter, and edge detection.
3. To implement color transformation operations such as grayscale, sepia, negative, and posterization.
4. To provide geometric transformation operations such as rotation, reflection, scaling, center crop, and custom crop.
5. To support audio processing operations such as volume adjustment, normalization, reverse, and splice/cut.
6. To process video files frame by frame by applying selected image processing operations to each frame.
7. To demonstrate multimedia processing concepts manually at the pixel, sample, and frame level instead of depending on ready-made filter functions.
8. To allow users to preview the processed result and save the final output.

Background Research

Multimedia processing is an important area in computer science that focuses on manipulating and analyzing different types of media such as images, audio, and videos. In this project, the system was designed to demonstrate several multimedia processing concepts through a practical desktop application.

Image processing is one of the main parts of the project. A digital image is made of pixels, and each pixel usually contains color values such as red, green, and blue. By changing these values, different effects can be applied to the image. For example, brightness adjustment changes the intensity of the pixels, while contrast adjustment changes the difference between dark and bright areas. Other operations such as filters and edge detection are used to enhance images or identify important visual details.

Color transformations are also used in the project to change the appearance of images. Examples include grayscale, sepia, negative, and posterization. These operations are useful for understanding how color values can be converted or reduced to create different visual results.

Geometric transformations focus on changing the position or size of pixels in an image. In this project, operations such as rotation, reflection, scaling, and cropping are included. These transformations demonstrate how image coordinates can be modified to create a new output image.

Audio processing is another important multimedia concept. Digital audio is represented as a sequence of samples. By modifying these samples, the system can apply operations such as volume adjustment, normalization, reverse, and cut/splice. This helps show how sound can be processed at the sample level.

Video processing can be understood as processing a sequence of image frames. In this project, video files are read frame by frame, and selected image processing operations are applied to each frame. This demonstrates the relationship between image processing and video processing. The project uses Python with several libraries to build and support the application.

Tools and Libraries Used

The project was implemented using Python because it is simple, flexible, and provides many useful libraries for multimedia processing. The system also uses several libraries to support the graphical user interface, image processing, audio

processing, and video processing. The main tools and libraries used in this project are:

1. CustomTkinter

CustomTkinter was used to build the graphical user interface. It provides a modern-looking GUI that allows users to upload files, choose processing operations, adjust sliders, preview results, and save the processed output [1].

2. Pillow

Pillow was used for opening, displaying, and saving image files. It helps the application handle image formats such as PNG, JPG, JPEG, and BMP [2].

3. NumPy

NumPy was used to store and manipulate pixel arrays and audio sample arrays. It makes it easier to work with numerical data such as RGB values and audio samples [3].

4. Wave Module

Python's built-in wave module was used to read and write WAV audio files. This allows the application to process WAV audio directly without requiring extra external tools [4].

5. PyDub

PyDub was used to support MP3 loading and exporting. However, MP3 support may require FFmpeg to be installed on the computer [5].

6. OpenCV

OpenCV was used for reading video files, extracting video frames, reading video metadata, and saving processed video files. In this project, OpenCV is used for video input and output only, while the actual image processing operations are implemented manually [6].

7. FFmpeg

FFmpeg may be required when working with MP3 audio files through PyDub. WAV files can still be processed without FFmpeg [7].

System Methodology

The system follows a simple workflow. First, the user opens the application and selects the type of processing, such as image, color, geometric, audio, or video processing. Then, the user uploads the required multimedia file and chooses the operation to apply. For image processing, the system modifies pixel values manually. For audio processing, the system modifies audio samples directly. For video processing, the system reads the video frame by frame and applies the selected image operation to each frame. After processing, the application displays a preview of the result when possible. Finally, the user can save the processed output to the computer. This workflow helps demonstrate multimedia processing concepts at the pixel, sample, and frame level.

MultiMedia Processing Studio - System Workflow

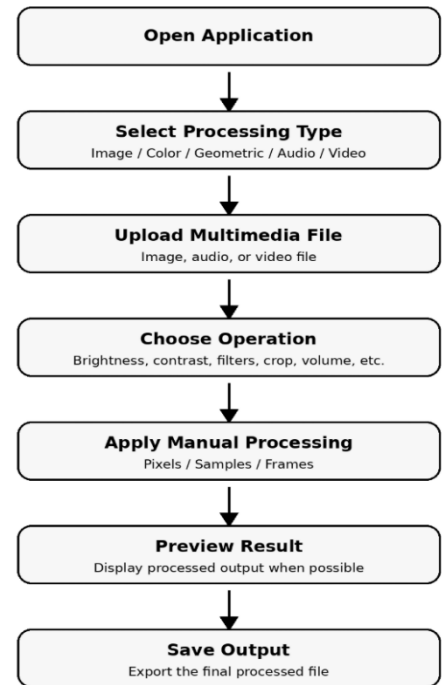


Figure 1: General workflow of the MultiMedia Processing Studio application.

System Design

The system was designed as a modular desktop application. The application is divided into several main parts: the graphical user interface, the processing modules, and the file utility functions.

The graphical user interface is responsible for allowing the user to upload files, choose the processing type, adjust values using sliders or input fields, preview the result, and save the output. The processing modules contain the main multimedia operations. Each module focuses on a specific type of processing. Image processing handles brightness, contrast, filters, and edge detection. Color transformation handles grayscale, sepia, negative, and posterization. Geometric transformation handles rotation, reflection, scaling, and cropping. Audio processing

handles volume, normalization, reverse, and cut. Video processing handles reading videos frame by frame and applying image operations to the frames. The system structure makes the project easier to understand, test, and modify because each part has a clear responsibility.

Implementation

The project was implemented as a Python desktop application with a modular structure. The main file is responsible for starting the application, while the GUI folder contains the graphical user interface. The processing folder includes separate modules for image processing, color transformations, geometric transformations, audio processing, and video processing.

The image processing module includes operations such as brightness adjustment, contrast adjustment, filters, histogram equalization, and edge detection. The color transformation module applies effects such as grayscale, sepia, negative, and posterization by manually changing RGB values.

The geometric transformation module handles operations such as rotation, reflection, scaling, and cropping by changing pixel positions. The audio processing module modifies audio samples directly to apply volume adjustment, normalization, reverse, and cut operations. For video processing, the system reads the video frame by frame and applies the selected image processing operation to each frame before saving the final output. This modular implementation makes the project easier to understand, test, and maintain.

User Manual

This section explains how to run and use the MultiMedia Processing Studio application. First, the user should open the project folder and install the required libraries by running the following command:

```
pip install -r requirements.txt
```

After installing the libraries, the application can be started by running:

```
python main.py
```

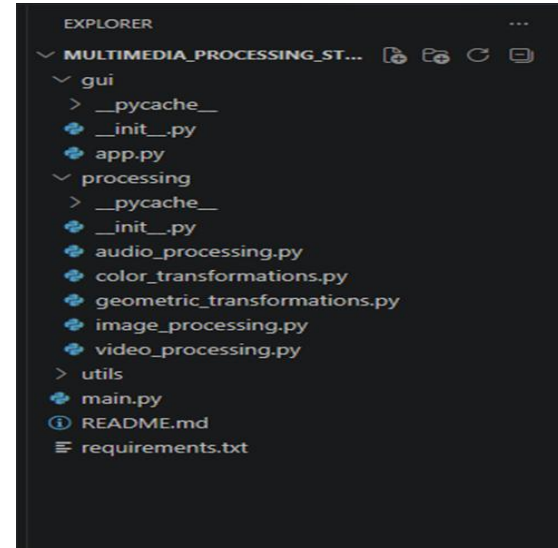


Figure 3: Project implementation structure showing the GUI folder, processing modules, utility folder, main file, README file, and requirements file.

When the application opens, the user can choose one of the available tabs, such as Image Processing, Color Transformations, Geometric Transformations, Audio Processing, or Video Frame Processing. The user then uploads a multimedia file, selects the required operation, adjusts the available controls such as sliders or input fields, and clicks the apply button to process the file.

For image operations, the original image and the processed image are displayed side by side. For audio operations, the user can apply changes such as volume, normalize, reverse, or cut, then save the output file. For video operations, the system shows video information and processes the video frame by frame when saving the result.

Finally, the user can save the processed output using the Save Result button.

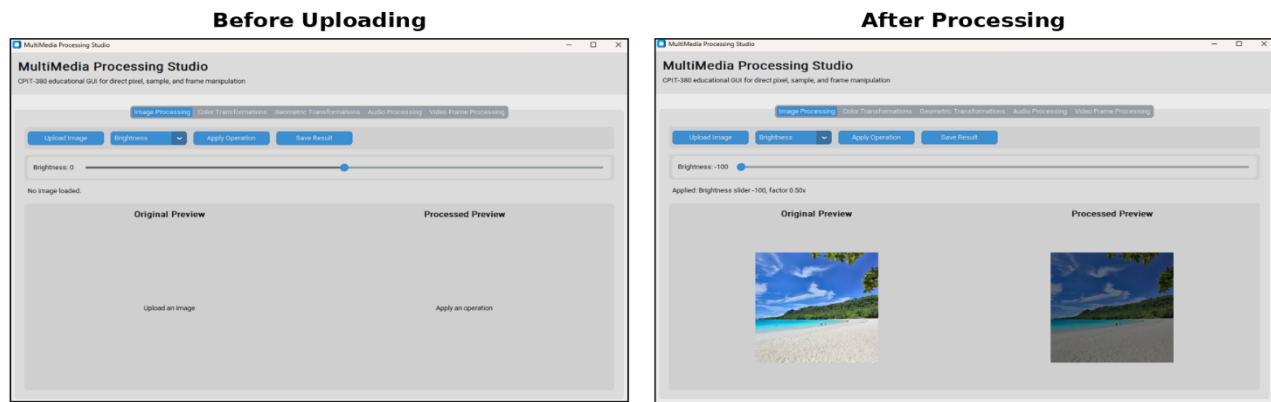


Figure 4: User manual screenshots showing the main interface before uploading a file (left) and an image processing example after applying brightness adjustment (right).

Testing and Results

The system was tested using different types of multimedia files to make sure that the main functions work correctly. The testing included image processing, color transformations, geometric transformations, audio processing, and video frame processing. For image processing, an image file was uploaded and operations such as brightness, contrast, and filters were applied. The application successfully displayed the original image and the processed image side by side.

For color and geometric transformations, operations such as grayscale, negative, rotation, reflection, scaling, and cropping were tested. The results showed that the system correctly changed the image colors, direction, size, or selected area based on the chosen operation. For audio processing, a short audio file was tested using volume adjustment, normalization, reverse, and cut. The application successfully processed the audio and allowed the user to save the output file. For video

processing, a short video file was tested. The system read the video frame by frame, applied the selected image operation, and saved the processed video successfully.

Table 1: System Testing Results

Test Case	Input	Operation	Expected Output	Result
1	Image file	Brightness adjustment	Image becomes darker or brighter	Pass
2	Image file	Grayscale	Image converted to grayscale	Pass
3	Image file	Rotation	Image rotated correctly	Pass
4	Audio file	Volume adjustment	Audio volume changes	Pass
5	Audio file	Reverse	Audio plays in reverse order	Pass
6	Video file	Brightness adjustment	Video frames processed successfully	Pass

Testing Screenshots

1.image processing

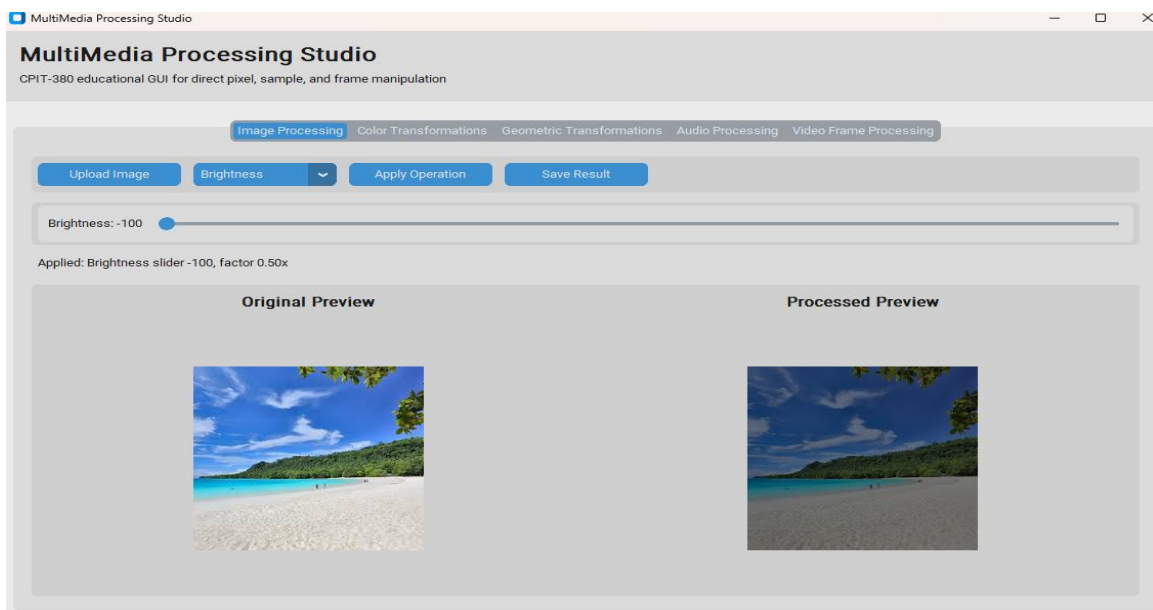


Figure 5: Brightness adjustment test showing the original image and the processed darker image.

2.color transformations

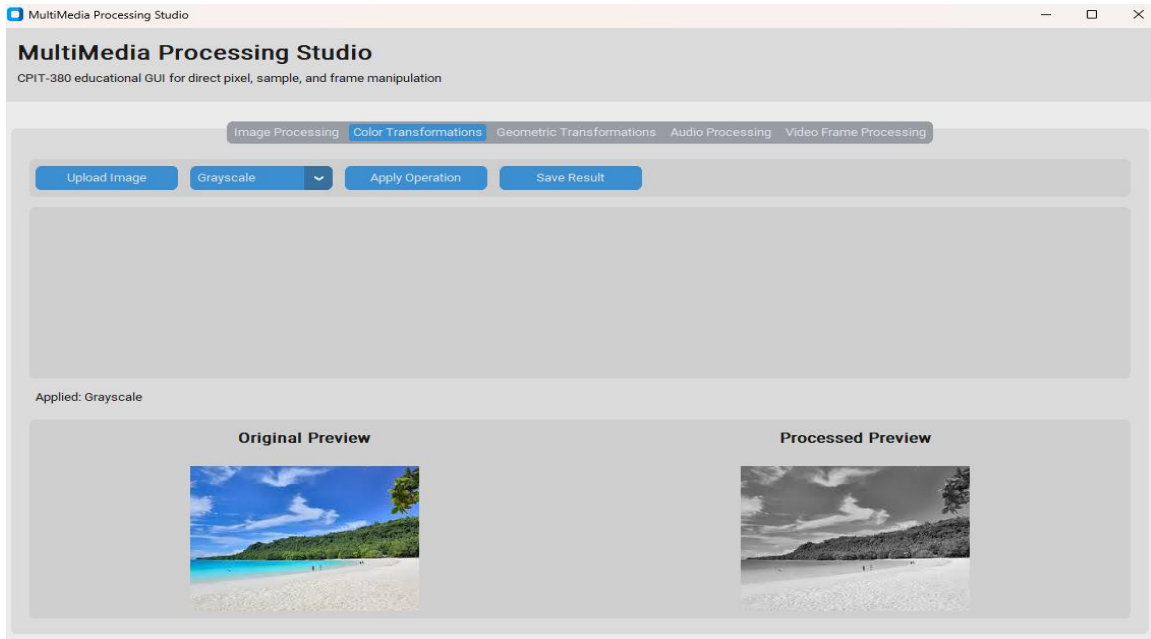


Figure 6: Color transformation test showing the original image and the grayscale processed image.

3. Geometric Transformations

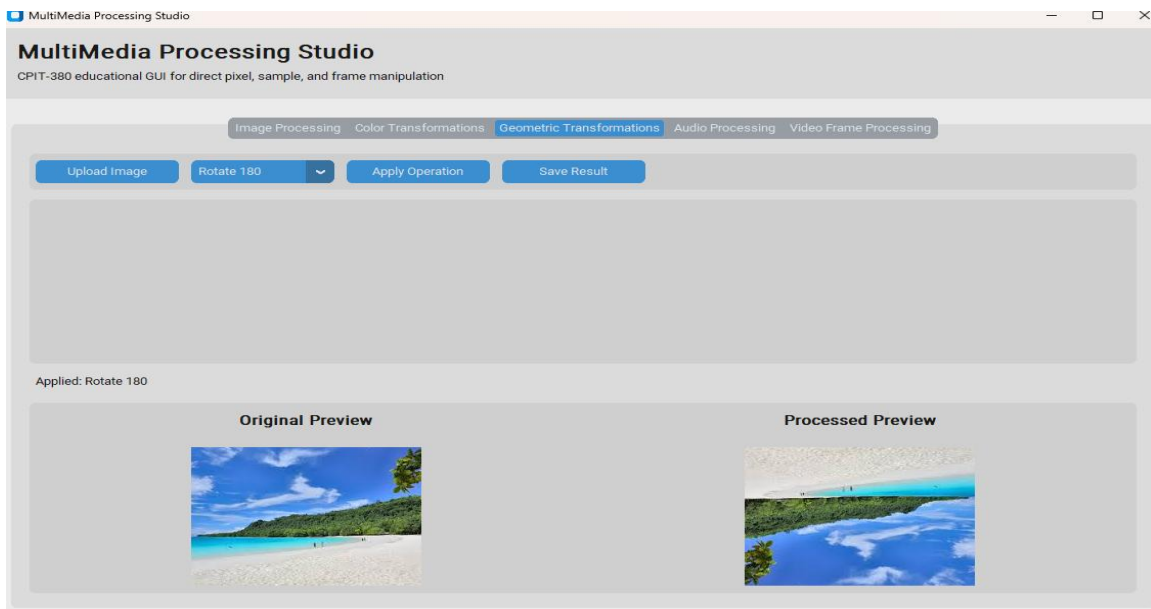


Figure 7: Geometric transformation test showing the original image and the processed image after applying 180-degree rotation.

4. Audio Processing

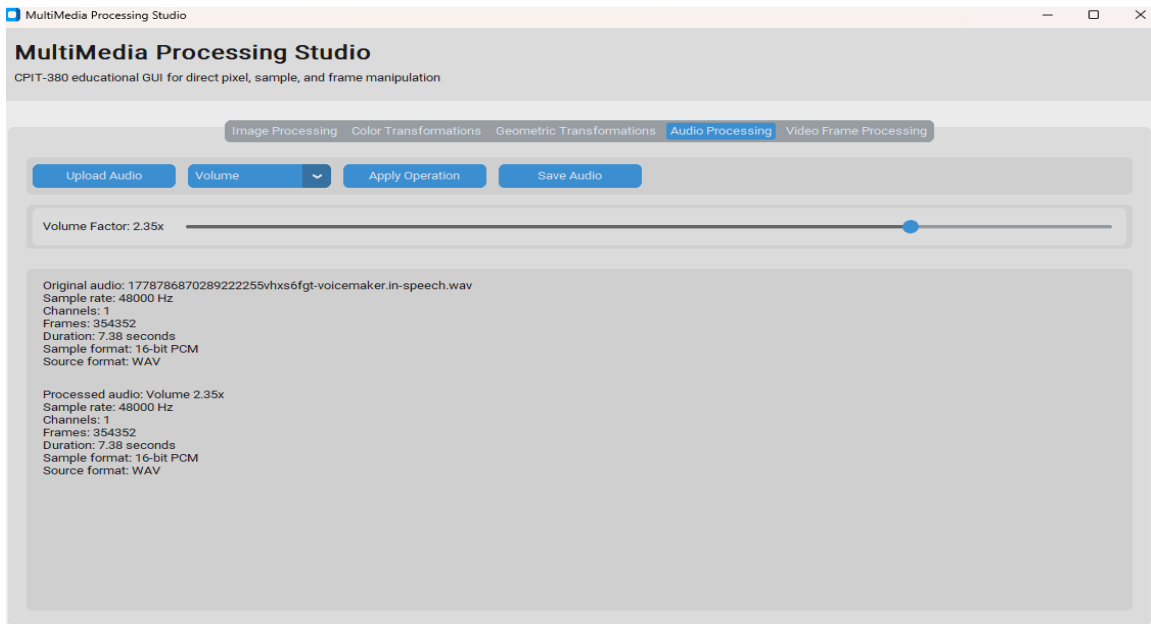


Figure 8: Audio processing test showing volume adjustment applied to a WAV audio file.

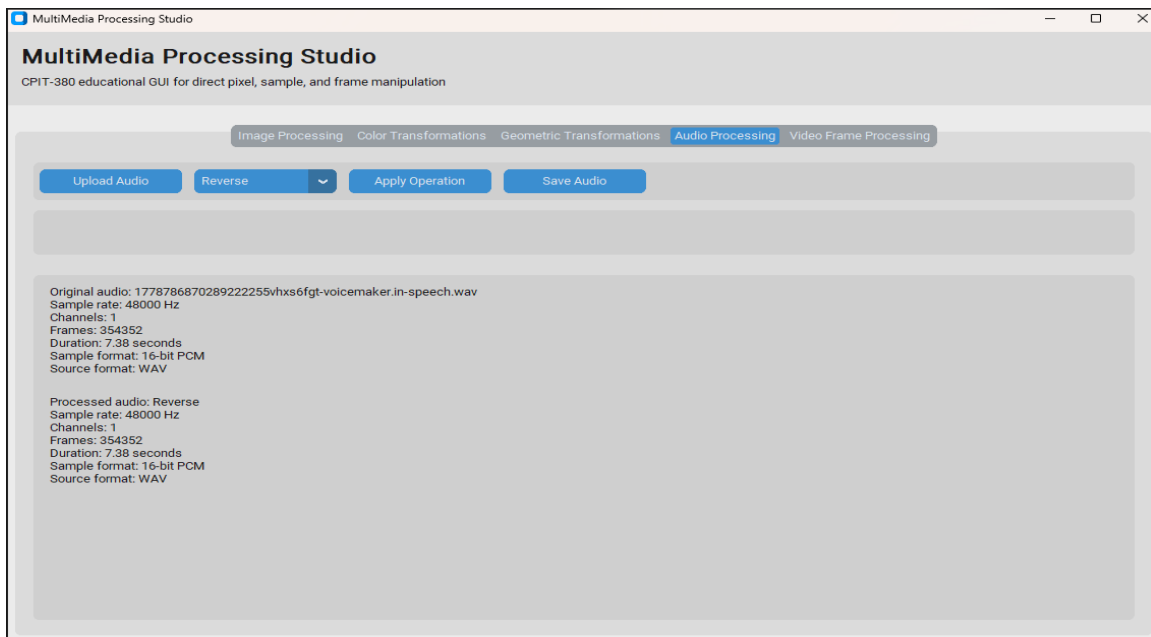


Figure 9: Audio processing test showing the reverse operation applied to a WAV audio file.

5. Video Frame Processing

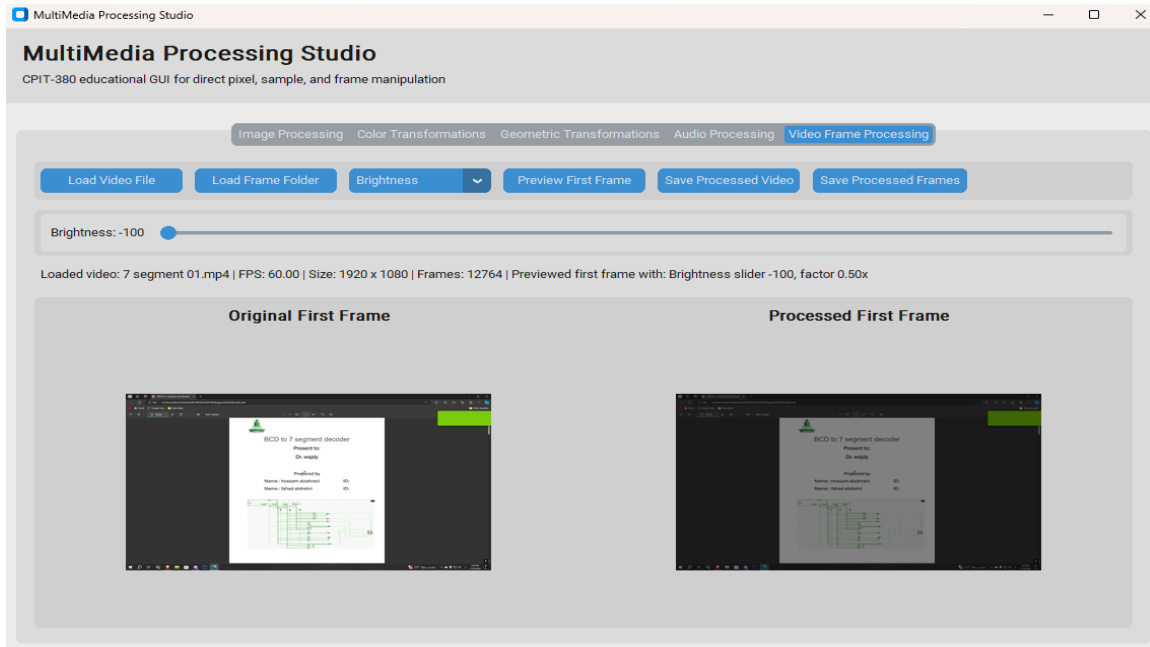


Figure 10: Video frame processing test showing the original first frame and the processed first frame after applying brightness adjustment.

Challenges and Limitations

During the implementation and testing of the project, several challenges and limitations were observed. One challenge was handling different multimedia file formats. For example, WAV audio files worked directly, while MP3 files may require additional support such as FFmpeg. Another challenge was video processing, because video files are processed frame by frame. This can take more time depending on the video length, resolution, and number of frames.

Some processing operations also have limitations. For example, rotation and scaling were not applied in the video tab because they may change the frame size and make video exporting less reliable. The application is designed for educational purposes, so it does not provide advanced editing tools like professional multimedia software. However, it successfully demonstrates the main CPIT-380 concepts by applying manual processing operations to pixels, audio samples, and video frames.

Conclusion

In conclusion, the MultiMedia Processing Studio project successfully achieved its main goal of creating an educational multimedia processing application. The system allows users to upload multimedia files, apply different processing operations, preview the results, and save the processed output. The project demonstrates several important CPIT-380 concepts, including image processing, color transformations, geometric transformations, audio processing, and video frame processing. The implementation focuses on manual processing at the pixel, sample, and frame level, which helps explain how multimedia data is represented and modified inside a computer system.

Although the application is not designed to replace professional editing software, it provides a practical and useful prototype for learning multimedia processing concepts.

References

- [1] CustomTkinter, "CustomTkinter Documentation," CustomTkinter, 2026. [Online]. Available: <https://customtkinter.tomschimansky.com/>. [Accessed may 2026].
- [2] Pillow, "Pillow Documentation," Read the Docs, 2026. [Online]. Available: <https://pillow.readthedocs.io/en/stable/>. [Accessed may 2026].
- [3] NumPy Developers, "NumPy Documentation," NumPy, 2026. [Online]. Available: <https://numpy.org/doc/>. [Accessed may 2026].
- [4] Python Software Foundation, "wave — Read and write WAV files," Python Software Foundation, 2026. [Online]. Available: <https://docs.python.org/3/library/wave.html>. [Accessed may 2026].
- [5] PyDub, "PyDub Documentation," PyDub, 2026. [Online]. Available: <https://www.pydub.com/>. [Accessed may 2026].
- [6] OpenCV, "OpenCV-Python Tutorials," OpenCV, 2026. [Online]. Available: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html. [Accessed may 2026].
- [7] FFmpeg, "FFmpeg Documentation," FFmpeg, 2026. [Online]. Available: <https://ffmpeg.org/documentation.htm>. [Accessed may 2026].